

UNIVERSITÀ
DI TORINO

DIPARTIMENTO DI
INFORMATICA

CORSO DI LAUREA
TRIENNALE

A.A. 2026/27

Canale B

Programmazione I

Teoria

Elvio Amparore

Memoria e Variabili

Modello della memoria per il linguaggio C, dalle variabili alle espressioni

Dai bit ai tipi di dati strutturati

- Il software serve a rappresentare e manipolare la realtà.
- Ogni informazione del mondo (numeri, testi, immagini, decisioni) deve essere trasformata in dati per essere elaborata da una macchina.
- Ma i dati da soli non bastano: occorre stabilire come interpretarli
- Questo è il ruolo dei **tipi**: regole che danno significato ai bit in memoria e ne delimitano l'uso.

*Un **tipo di dato** è un ponte tra i bit e il senso che vogliamo dare ai dati codificati tramite quei bit. Senza i tipi, avremmo solo sequenze di bit indistinguibili.*

I tipi nel linguaggio C

Un **tipo** qualifica una sequenza di bit secondo varie dimensioni:

- Nome: identificatore usato nel codice (int, float, struct Persona)
- Dimensione in memoria: quanti byte occupa (int = tipicamente 4B, double = 8B)
- Dominio di valori: insieme di valori validi (es. int = numeri interi compresi in un certo intervallo)
- Operazioni consentite: quali operatori o funzioni si possono applicare (somma su interi, sqrt su float, …)
- Rappresentazione interna: come i bit vengono codificati (complemento a 2 per interi, IEEE 754 per float, …) → ve ne occuperete in *Architetture degli Elaboratori* e in *Fondamenti dell'Informatica*.
- Compatibilità con altri tipi: regole di conversione implicite ed esplicite (cast)

Le variabili in C

- Una **variabile** associa un nome e un tipo di dato a una porzione di memoria che contiene un valore.
- Permette al programmatore di lavorare con i dati attraverso nomi simbolici, senza dover manipolare direttamente indirizzi di memoria e rappresentazioni binarie.
- Nel modello EDVAC/Von Neumann le operazioni agiscono direttamente su valori memorizzati a determinati indirizzi. I linguaggi di alto livello introducono le variabili come astrazione di queste posizioni di memoria.
- Possiamo così esprimere le operazioni sui dati in modo più leggibile e indipendente dai dettagli fisici della loro collocazione in memoria.

Le variabili in C

Una variabile è definita da:

- **Nome**: identificatore scelto dal programmatore (contatore, altezza, media)
- **Tipo**: stabilisce il significato dei bit (es. int → intero, float → numero reale)
- **Indirizzo**: la posizione fisica in memoria dove il dato è memorizzato.
- **Valore**: il contenuto attuale, che può cambiare durante l'esecuzione.

```
int contatore = 1;
```

nome: contatore | *indirizzo*: 0x904A | *valore*: 1 | *tipo*: int

```
float media = 60.54;
```

nome: media | *indirizzo*: 0x367F | *valore*: 60.54 | *tipo*: float

Le variabili in C

- Una volta dichiarata una **variabile**, il programmatore normalmente non specifica il suo indirizzo di memoria: la collocazione concreta viene determinata automaticamente* dal compilatore e dal sistema di esecuzione.
- Il programmatore stabilisce invece il tipo di dato, quindi:
  - quali valori la variabile può rappresentare;
  - come tali valori sono codificati in memoria;
  - quali operazioni possono essere applicate;
  - come valori di tipi diversi possono interagire.
- Comprendere i tipi significa quindi collegare il livello astratto del C alla rappresentazione concreta dei dati nella memoria della macchina.

** La gestione della memoria in C è solo parzialmente automatica.*

Panoramica del type system del C

Tipi primitivi (built-in)

Numeri Interi (ℤ)
int, short, char, ...

Numeri Naturali (ℕ)
unsigned int,
unsigned short, ...,
bool

Numeri Reali (ℝ)
float, double

Tipi incompleti
void

Tipi derivati

Puntatori

Array

Struct

Union

Enum

Funzioni

Typedef

Tipi qualificati

const volatile restrict

Tipi primitivi del linguaggio C

Partiamo da un sottoinsieme dei *tipi primitivi*, utili per iniziare:

- **int**: tipo delle variabili che rappresentano numeri interi ($\mathbb{Z}$)
  - Altri tipi interi: short int, long int, long long int
- **unsigned int**: tipo che rappresenta numeri naturali ($\mathbb{N}$)
- **float**: tipo dei numeri reali ($\mathbb{R}$)
  - Altri tipi reali: double, long double
- **char**: tipo dei caratteri, occupa 1 byte (8 bit)
- **size_t**: tipo intero unsigned delle dimensioni.
  - definito in <stddef.h>
- **bool**: tipo delle operazioni logiche. 0/1, false/true
  - definito in <stdbool.h>

I tipi numerici del C sono finiti

Le categorie $\mathbb{N}$, $\mathbb{Z}$ e $\mathbb{R}$ indicano il dominio matematico di riferimento, ma non coincidono con i tipi del C.

- Gli insiemi $\mathbb{N}$, $\mathbb{Z}$ e $\mathbb{R}$ sono infiniti.
- Un valore C è rappresentato usando una quantità finita di memoria, quindi ogni tipo può rappresentare solo un insieme finito di valori.
- **int** rappresenta solo un intervallo finito di numeri interi.
- **unsigned int** rappresenta solo un intervallo finito di numeri naturali.
- **float** e **double** rappresentano un insieme finito di valori usati per approssimare i numeri reali.

I tipi numerici del C sono quindi modelli finiti dei corrispondenti domini matematici. Questo introduce limiti di intervallo per gli interi e di precisione per i numeri float/double.

Dichiarazione di variabili

- Ogni variabile deve essere dichiarata con un tipo e un identificatore

```
int x;
```

- Questo riserva lo spazio necessario in base al tipo in un'area di memoria detta **pila** (stack)
- Dopo la dichiarazione, l'area di memoria riservata diventa accessibile usando il nome scelto
- L'identificatore è *simbolico* perché rappresenta un indirizzo della memoria, e non esiste nell'assembly

Stampa dei valori

Vediamo una prima operazione che si può fare sulle **variabili** di un tipo primitivo: stamparne il **valore** a schermo.

```
1.  int x = 10;         // variabile intera
2.  float y = 3.14f;    // variabile reale (o in “virgola mobile”)
3.  // Stampa dei valori
4.  printf("La variabile x vale: %d\n", x);    // %d → intero (int)
5.  printf("La variabile y vale: %f\n", y);    // %f → float (decimale)
```

La funzione printf permette di stampare testo assieme ai valori delle variabili: `printf("testo e specificatori", lista_di_variabili);`

- La stringa di formato contiene testo normale più specificatori di formato (es. %d, %f, %c ...)
- Dopo la stringa, si passano come argomenti le variabili da stampare, nell’ordine corrispondente agli specificatori.

Tassonomia dei tipi primitivi del C

	Interi (ℤ)	Naturali (ℕ)	Reali (ℝ)
Numeri	char	unsigned char	float
	short short int	unsigned short unsigned short int	double
	int signed int	unsigned int unsigned signed int	
	long long int	unsigned long unsigned long int	
	long long long long int	unsigned long long unsigned long long int	
Dimensioni	ptrdiff_t	size_t	
Carattere	char		
Byte		unsigned char	
Logico	bool		

Dimensione delle variabili

Il C non definisce in modo rigido la dimensione in bits/bytes dei tipi primitivi (per portabilità su architetture molto diverse), ma definisce relazioni di ordine tra le dimensioni:

$$\text{sizeof(char)} \leq \text{sizeof(short)} \leq \text{sizeof(int)} \leq \text{sizeof(long)} \leq \text{sizeof(long long)}$$

$$\text{sizeof(float)} \leq \text{sizeof(double)}$$

Il risultato dell'operatore sizeof ha tipo size_t.

Consideriamo alcune configurazioni tipiche delle dimensioni dei tipi sulle piattaforme a 32 e 64 bit, che useremo come riferimento (slide successiva)

Tabella dei tipi e specificatori di conversione

Dimensioni e specificatori dei tipi elementari con segno:

Tipo	Uso	CPU 32 bit		CPU 64 bit		specificatori di conversione
		Dimensione [Byte / bit]	Intervallo	Dimensione [Byte / bit]	Intervallo	
char	ASCII	1 / 8	-128, 127	1 / 8	-128, 127	%c
short short int	numeri **interi**	2 / 16	-2^{15}, $+2^{15}-1$	2 / 16	-2^{15}, $+2^{15}-1$	%hd
int signed int		4 / 32	-2^{31}, $+2^{31}-1$	4 / 32	-2^{31}, $+2^{31}-1$	%d
long long int		4 / 32 8 / 64	-2^{31}, $+2^{31}-1$ -2^{63}, $+2^{63}-1$	8 / 64	-2^{63}, $+2^{63}-1$	%ld
long long long long int		8 / 64	-2^{63}, $+2^{63}-1$	8 / 64	-2^{63}, $+2^{63}-1$	%lld
ptrdiff_t		4 / 32	-2^{31}, $+2^{31}-1$	8 / 64	-2^{63}, $+2^{63}-1$	%t
float	numeri **reali** (con virgola)	4 / 32	10^{-38}, 10^{+38}	4 / 32	10^{-38}, 10^{+38}	%f
double		8 / 64	10^{-308}, 10^{+308}	8 / 64	10^{-308}, 10^{+308}	%lf

Tabella dei tipi e specificatori di conversione

Dimensioni e specificatori dei tipi elementari senza segno:

Tipo	Uso	CPU 32 bit		CPU 64 bit		specificatori di conversione
		Dimensione [Byte / bit]	Intervallo	Dimensione [Byte / bit]	Intervallo	
unsigned char	bytes	1 / 8	**0, 255**	1 / 8	**0, 255**	%hhu
unsigned short unsigned short int	numeri **naturali**	2 / 16	**0, $+2^{16}-1$**	2 / 16	**0, $+2^{16}-1$**	%hu
unsigned int unsigned signed int		4 / 32	**0, $+2^{32}-1$**	4 / 32	**0, $+2^{32}-1$**	%u
unsigned long unsigned long int		4 / 32 8 / 64	**0, $+2^{32}-1$** **0, $+2^{64}-1$**	8 / 64	**0, $+2^{64}-1$**	%lu
unsigned long long unsigned long long int		8 / 64	**0, $+2^{64}-1$**	8 / 64	**0, $+2^{64}-1$**	%llu
size_t		4 / 32	**0, $+2^{32}-1$**	8 / 64	**0, $+2^{64}-1$**	%zu
float	numeri reali (con virgola)	4 / 32	10^{-38}, 10^{+38}	4 / 32	10^{-38}, 10^{+38}	%f
double		8 / 64	10^{-308}, 10^{+308}	8 / 64	10^{-308}, 10^{+308}	%lf

NOTA: non serve imparare a memoria queste tabelle!!

Identificatori delle variabili

- Il nome della variabile deve seguire regole precise:
  - Deve essere univoco all'interno del suo ambito (*scope*)
  - Deve iniziare con una lettera o underscore (spesso si preferisce la lettera minuscola)
  - Non può contenere spazi o simboli speciali
  - Può contenere lettere, numeri (non iniziali) e underscore

- Esempio: `int laMiaPrimaVariabile;`

Identificatori delle variabili

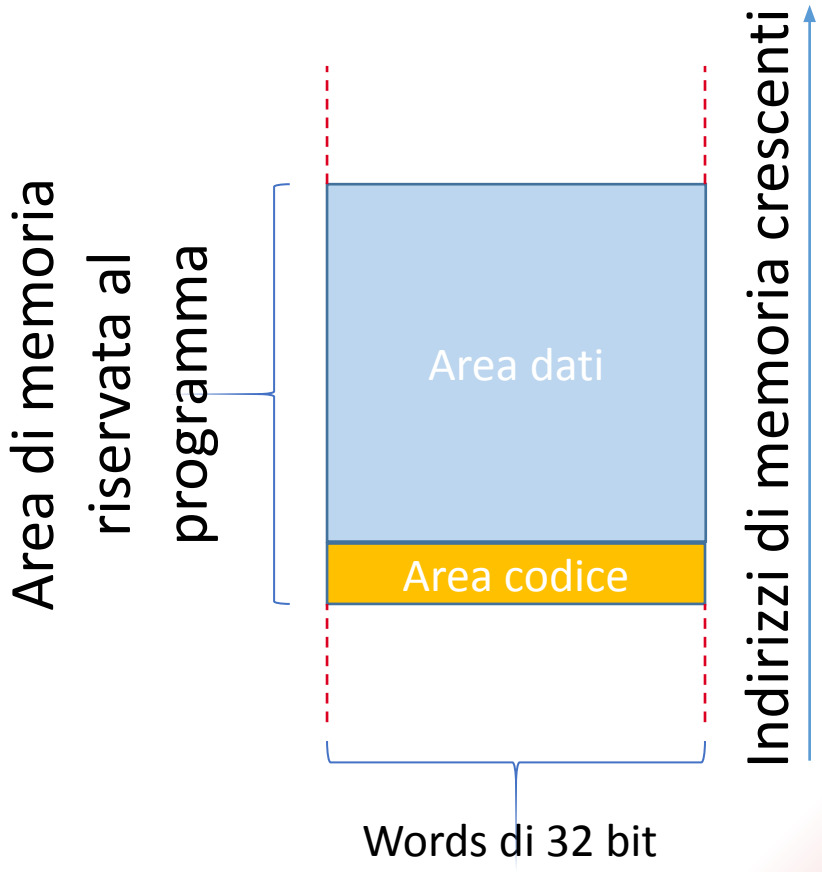
- Useremo delle convenzioni per gli identificatori delle variabili:
  - preferire il solo minuscolo, oppure per nomi composti da più parole si può usare la convenzione camelCase:
    `sommaTot`, `media7giorni`, `elementoTrovato`.
  - usare nomi significativi, che ne descrivano il ruolo:
    - **Buoni esempi**: `somma`, `media`, `massimoTrovato`
    - **Solo se è chiaro dal contesto**: `s`, `m`, `flag`
    - **Cattivi esempi**: `pippo`, `pluto`, `nonSoCheNomeDare`
- La scelta del nome di una variabile non è un dettaglio secondario, ma un elemento importante della prassi della programmazione
- Serve a rendere il codice leggibile e comprensibile: descrive chiaramente il significato del dato, riduce gli errori e facilita il ragionamento

Visibilità e durata delle variabili

- Una variabile può essere usata dove è **visibile** (*in scope*):
  - **Locale/Automatica** (default): dichiarata dentro una funzione o un blocco di codice **{ … }**, è accessibile solo lì
  - **Globale**: dichiarata fuori da ogni funzione, è visibile in tutto il file (e, se extern, anche in altri file).
  - **Statica**: variabile persistente ma visibilità nel blocco in cui compare.
- Una variabile esiste in memoria in funzione della sua **durata** (*lifetime*):
  - Le variabili **locali/automatiche** sono allocate sullo **stack**
  - Le variabili globali/statiche esistono per tutta la durata del programma
  - Le variabili **dinamiche** sono allocate nello **heap** (vedremo più avanti) con funzioni come malloc(), e persistono finché non vengono liberate manualmente.
- Per utilizzare il linguaggio C serve comprendere come la memoria è gestita (*allocata* e *deallocata*). Studieremo l'evoluzione della memoria dati del programma usando un **modello della memoria**

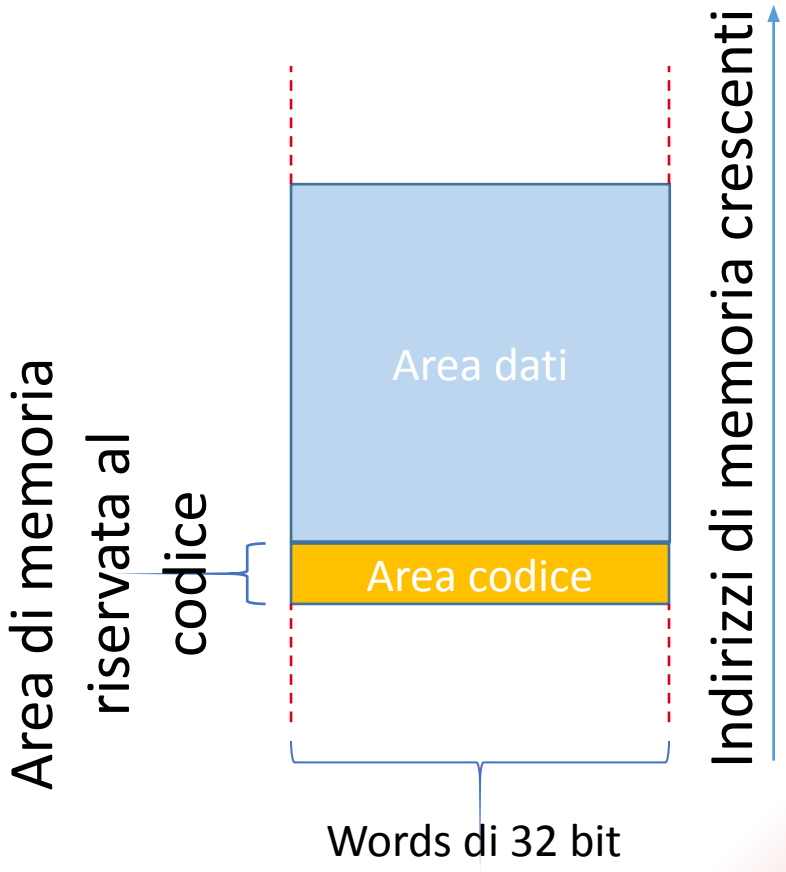
Un primo modello di memoria

- Al caricamento (loading) del programma dalla memoria secondaria, viene allocata un'area di memoria per codice e dati del programma



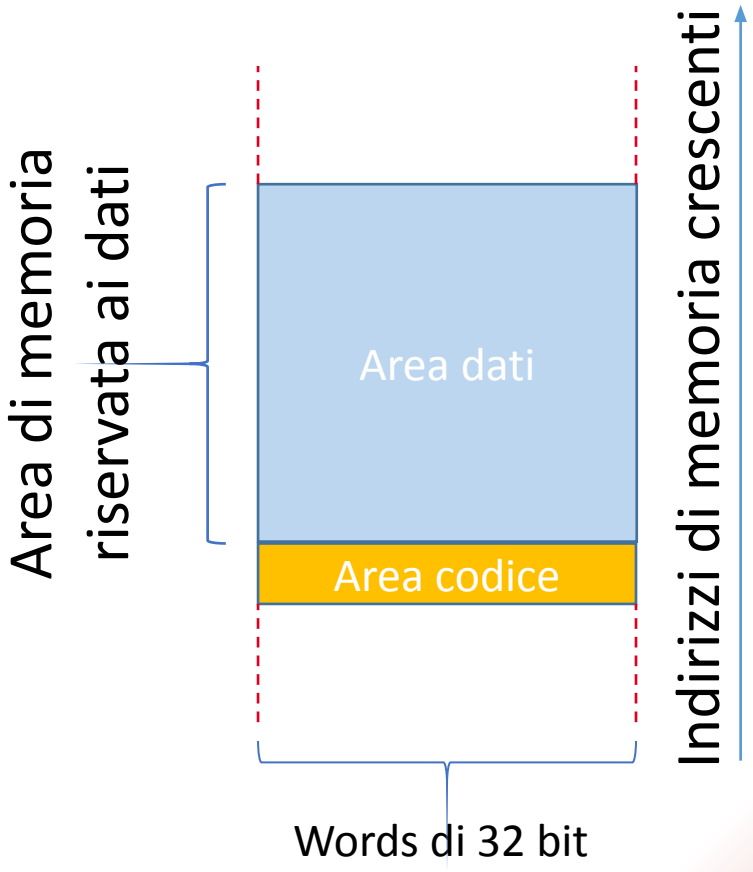
Un primo modello di memoria

- Al caricamento (loading) del programma dalla memoria secondaria, viene allocata un'area di memoria per codice e dati del programma
- La dimensione dell'area codice è fissa



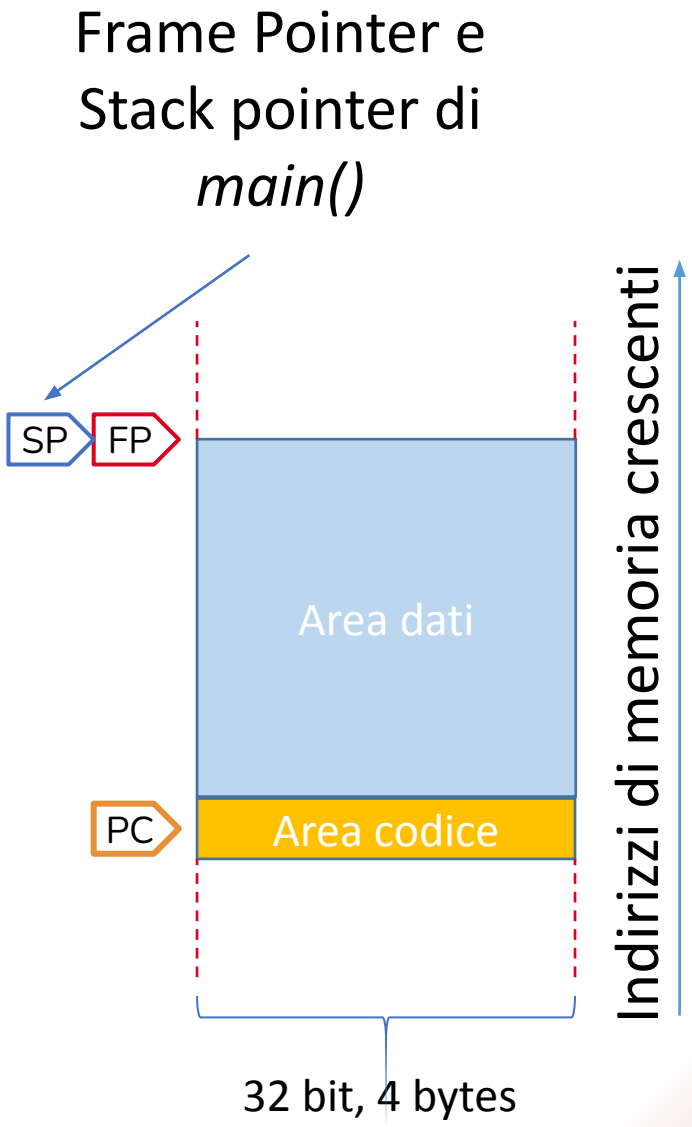
Un primo modello di memoria

- Al caricamento (loading) del programma dalla memoria secondaria, viene allocata un'area di memoria per codice e dati del programma
- La dimensione dell'area codice è fissa
- L'area dati è flessibile ma assumiamo abbia una dimensione *massima*
- Le variabili sono allocate nell'area riservata ai dati una dopo l'altra in ordine di dichiarazione



Un primo modello di memoria

- Il *Program Counter* (PC) parte nell'area codice, all'indirizzo di inizio del **main**
- Il *Frame Pointer* (FP) è l'indirizzo base da cui iniziare ad allocare le variabili del programma (partendo dal **main**)
- Lo *Stack Pointer* (SP) indica l'indirizzo a cui allocare la prossima variabile
- All'inizio, FP e SP coincidono

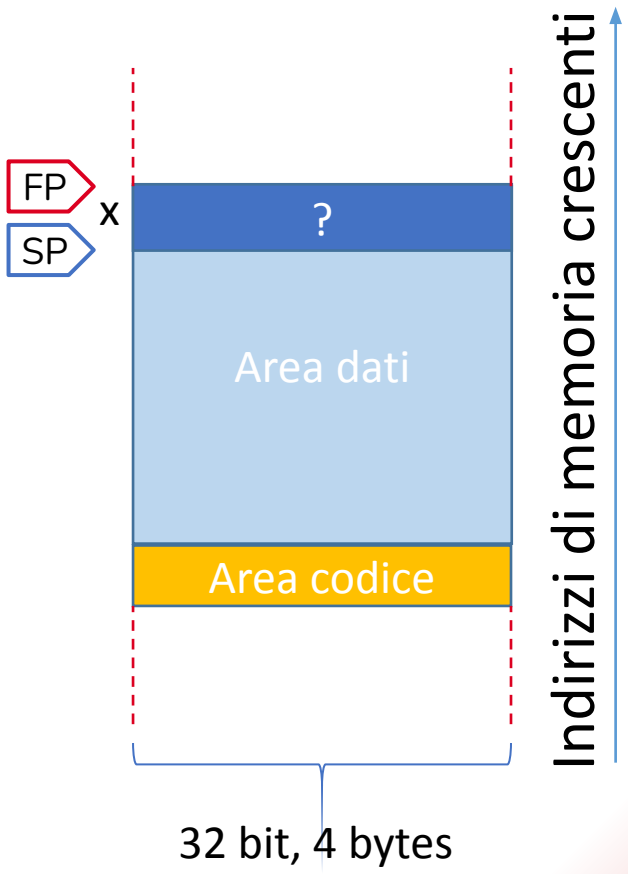



```
int main (void) {
    int x;
}
```

Un primo modello di memoria

- Chiamiamo **stack** (*pila*) la parte “alta” dell’area dati.
- Nel nostro modello rappresentiamo lo stack nella parte alta dell’area dati e assumiamo che cresca verso indirizzi più bassi.
  - NOTA: è una convenzione di rappresentazione: la direzione di crescita dello stack dipende dall’architettura e dal sistema.

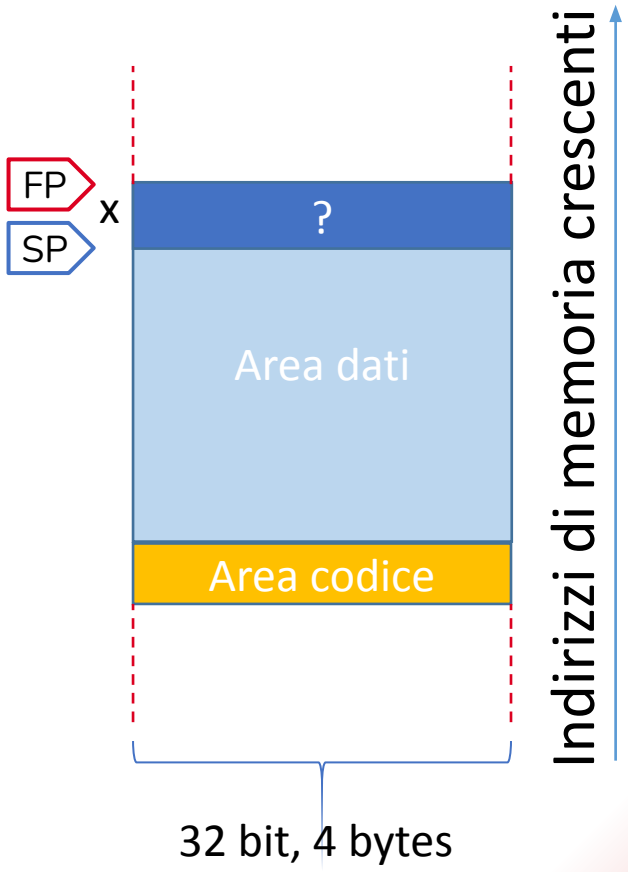
```
int main (void) {
    int x;
}
```



Un primo modello di memoria

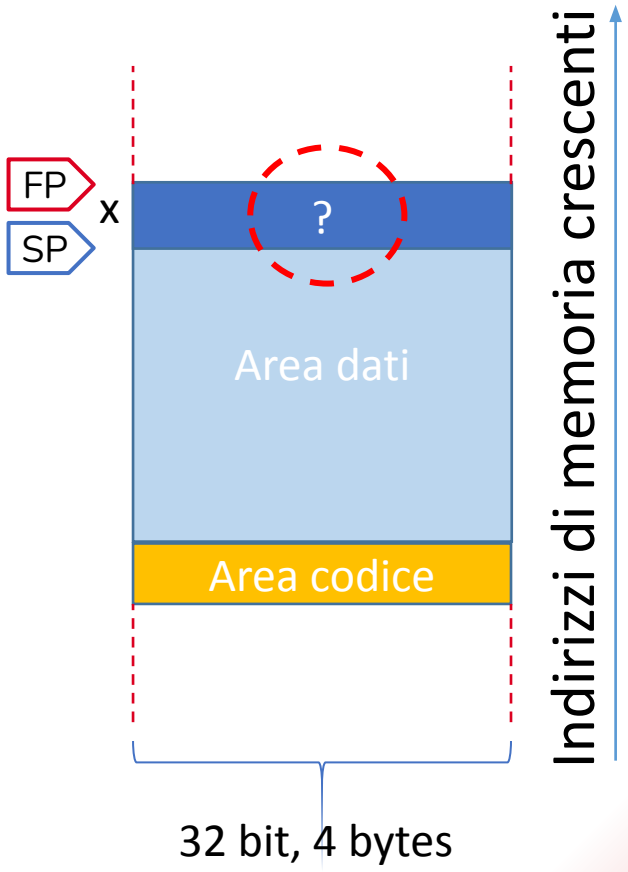
- Con questa convenzione, dopo la dichiarazione di x, SP viene decrementato della dimensione necessaria alla variabile (ad es. assumiamo 4 byte per il tipo int).
- L'allocazione di una variabile occupa memoria sullo stack.
  - Se lo spazio disponibile per lo stack si esaurisce, si può ottenere un errore runtime di tipo stack overflow.

```
int main (void) {
    int x;
}
```



Un primo modello di memoria

- Dichiarare una variabile locale senza inizializzarla riserva lo spazio di memoria, ma non le assegna un valore: il suo valore iniziale è indeterminato.
- Ne indicheremo il contenuto con «**?**».



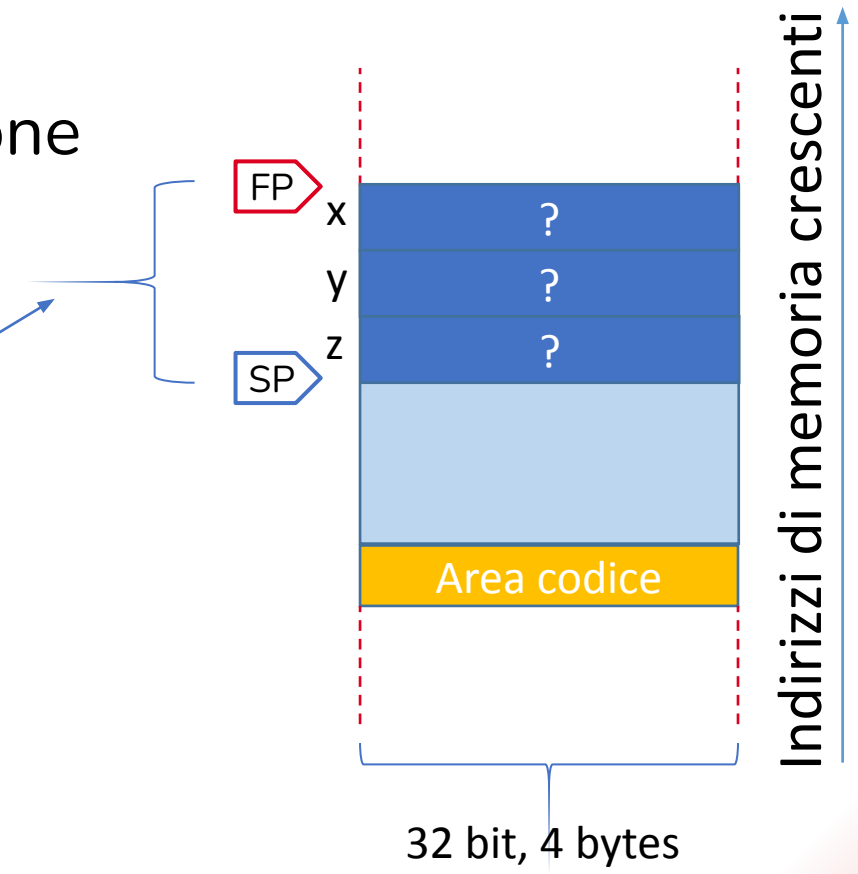

```
int main (void) {
    int x;
}
```

Un primo modello di memoria

- Per ogni variabile dichiarata, viene allocata una quantità di memoria corrispondente al tipo
- L'allocazione avviene in ordine di dichiarazione

```
int main(void) {
    int x;
    int y;
    int z;
}
```

12 bytes allocati per 3 variabili di tipo int da 4 bytes l'una



Assegnare valori

Vediamo una seconda operazione che si può fare sulle variabili di un tipo primitivo: **assegnargli un valore** (*assignment*).

- Dichiarata una variabile, è possibile scrivere un valore in tale area di memoria *assegnandogli* un valore con l'operatore «=»

```
int x;
x = 10;
```

- Dichiarazione e assegnamento possono essere eseguiti sulla sulla stessa linea, per semplicità (*definizione*)

```
int x = 10;
```

Assegnare valori

- L'assegnamento di valore deve riflettere il tipo della variabile

```
int x = -8;
```

- Alcuni valori danno overflow non rilevabili dal compilatore

```
int x = 2 147 483 649;
```

```
$ ./overflow
x vale -2147483647
```

- Alcune espressioni potrebbero essere interpretate diversamente da come inteso dal programmatore a seconda del tipo

```
int x = 8.89;
```

```
$ ./float2int
x vale 8
```

Assegnamento in C

- La forma generale di un assegnamento è:

  variabile **=** *espressione*;

- **Lato sinistro** (*lvalue*)

  variabile: indica l'area di memoria in cui salvare il risultato

- **Lato destro** (*rvalue*)

  espressione: un valore o una formula da calcolare

- Passaggi logici:
  - L'espressione a destra viene valutata (*evaluated*)
  - Se la valutazione ha successo, produce un valore
  - Questo valore viene copiato nell'area di memoria associata al nome della variabile

Espressioni aritmetiche in C (su interi)



- Per ora ci concentriamo solo su espressioni algebriche con variabili intere
- Il linguaggio delle espressioni aritmetiche elementari è:
  - **Somma**: `valoreA + valoreB`
  - **Sottrazione**: `valoreA - valoreB`
  - **Moltiplicazione**: `valoreA * valoreB`
  - **Divisione intera** (solo parte intera): `valoreA / valoreB`
  - **Modulo** (resto della divisione): `valoreA % valoreB`
  - **Parentesi tonde**: raggruppamento delle operazioni
- Esempi

```
int incasso = biglietti * prezzoBiglietto;
int utile = incasso - costoSala;
```

Precedenza degli operatori aritmetici

- Le regole di **precedenza** stabiliscono quale operatore viene valutato per primo quando in un'espressione sono presenti più operatori.
- Gli operatori *, /, % hanno la precedenza più alta tra quelli aritmetici
- Subito dopo, in ordine decrescente, troviamo + e -
- A parità di precedenza, le operazioni vengono valutate da sinistra a destra
- L'operatore di assegnamento = viene eseguito solo alla fine
- Le parentesi tonde () hanno precedenza assoluta:
  - permettono di forzare l'ordine di valutazione
  - possono essere annidate su più livelli

Divisione intera e modulo in C

- La **divisione intera** (/) restituisce solo la parte intera del quoziente, mentre la parte decimale viene scartata (non arrotondata)

```
int a = 13, b = 3;
int quot = a / b;     // quot = 4
```

- Il **modulo** (%) restituisce il resto della divisione intera

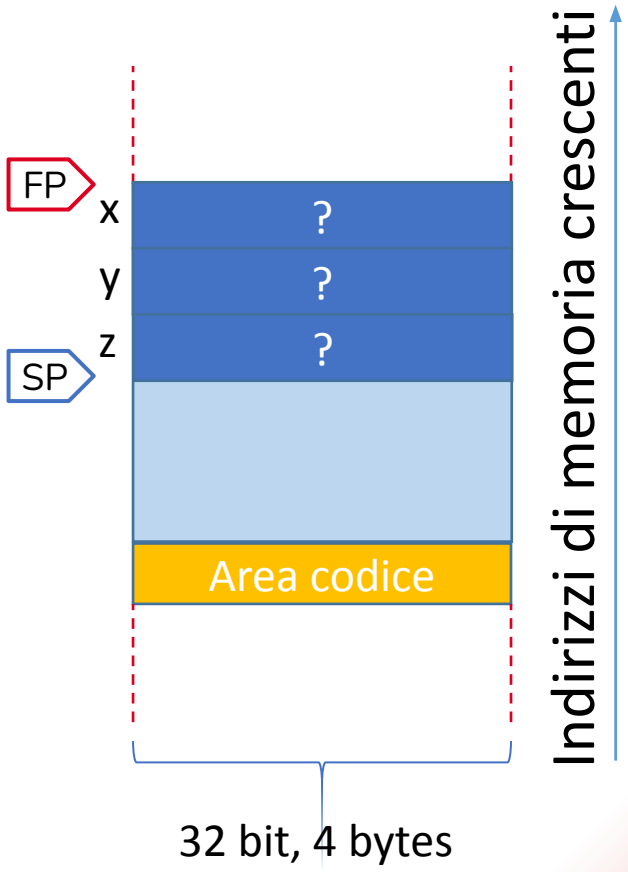
```
int r = a % b;    // r = 1 perchè 13 = 3*4 + 1
```

- Il modulo è molto utile per controllare la parità di un numero
  - se `num % 2` vale 0 allora `num` è **pari**
  - se `num % 2` vale 1 allora `num` è **dispari**

Espressioni algebriche

```
int main(void) {
    int x;
    int y;
    int z;
    x = 5;
    y = 2;
    z = x + y;
    printf("Le var x, y  e z valgono %d, %d, %d\n",
            x, y, z);
}
```

```
$ ./treVariabili
```

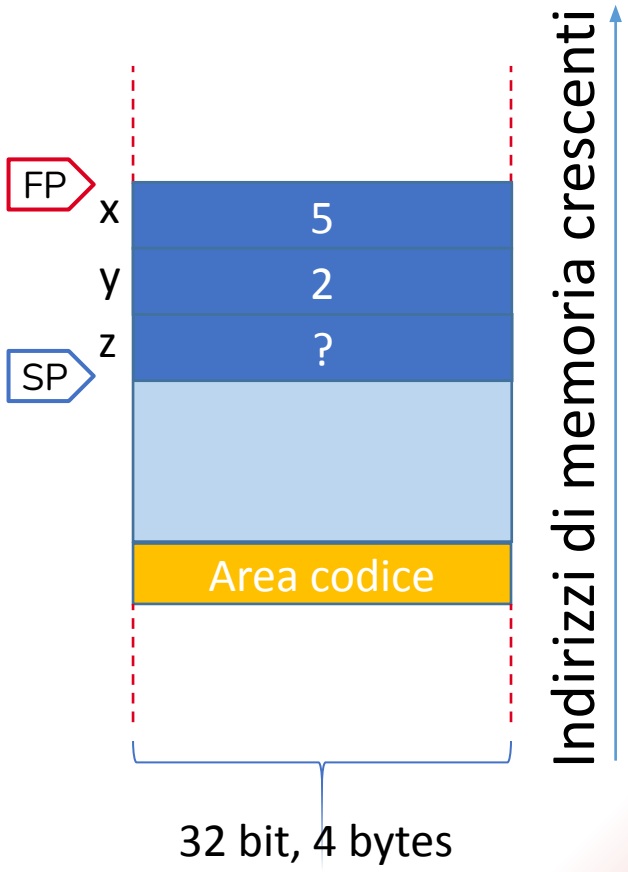


Espressioni algebriche

```
int main(void) {
    int x;
    int y;
    int z;
    x = 5;
    y = 2;
    z = x + y;
    printf("Le var x, y  e z valgono %d, %d, %d\n",
            x, y, z);
}
```

Assegnamento di un valore numerico

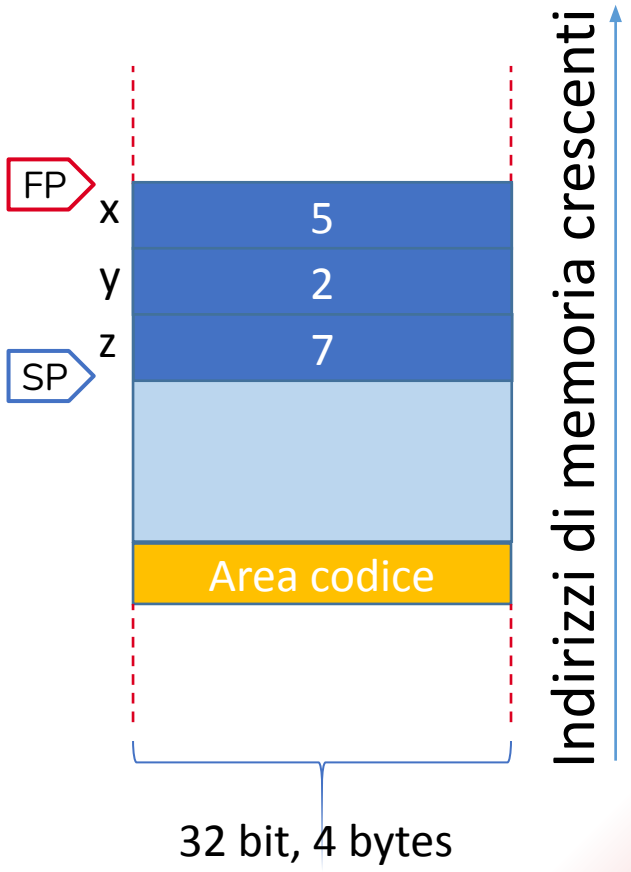
```
$ ./treVariabili
```



Espressioni algebriche

```
int main(void) {
    int x;
    int y;
    int z;
    x = 5;
    y = 2;
    z = x + y;
    printf("Le var x, y  e z valgono %d, %d, %d\n",
           x, y, z);
}
```

Assegnamento del risultato di espressione algebrica



LOAD, R0, @x
LOAD, R1, @y
ADD, R0, R1
STORE, R0, @z

```
$ ./treVariabili
```

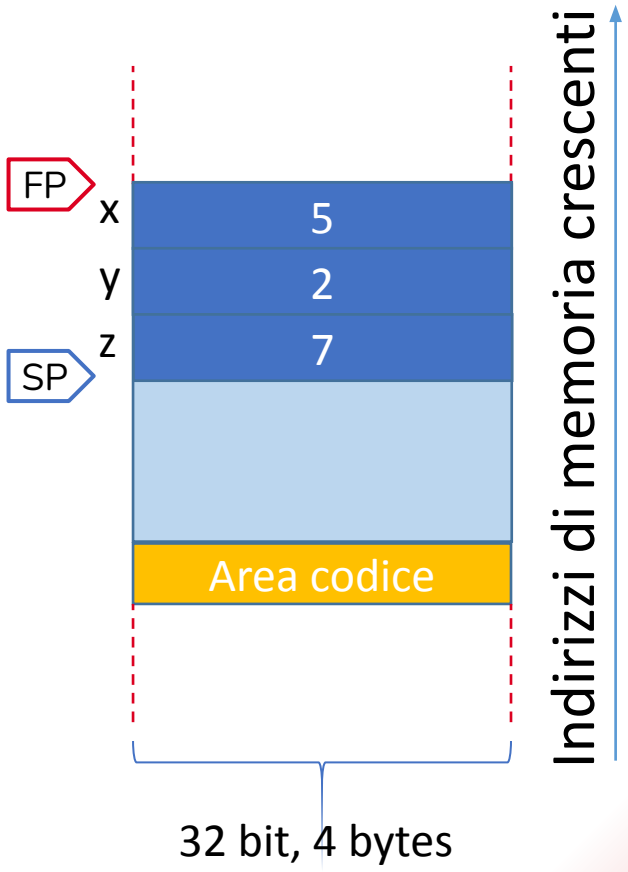
Espressioni algebriche

```
int main(void) {
    int x;
    int y;
    int z;
    x = 5;
    y = 2;
    z = x + y;
    printf("Le var x, y  e z valgono %d, %d, %d\n",
           x, y, z);
}
```

Stampa a schermo tre variabili

parametri separati da virgola

```
$ ./treVariabili
Le var x, y  e z valgono 5, 2, 7
```

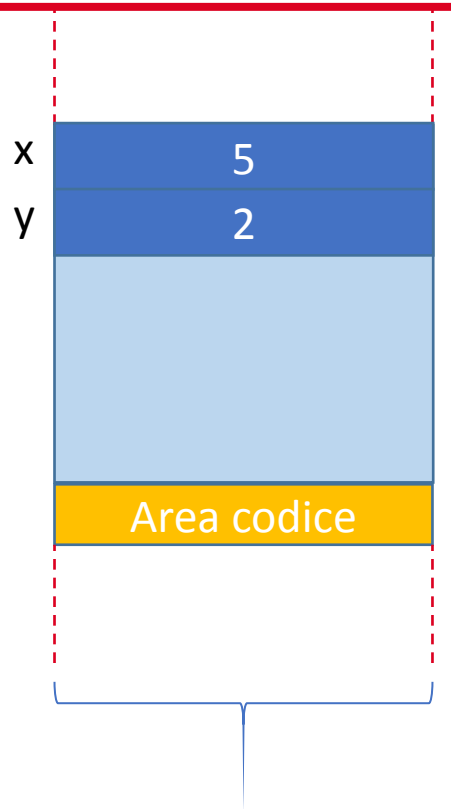


Altri esempi con printf

- Permette la stampa di variabili usando gli *specificatori di formato* (o "*di conversione*")
- Per gli int lo specificatore é **%d**
- Segue un *parametro* per ciascun specificatore

```
printf("x vale %d, y vale %d\n", x, y);
```

- Il primo *specificatore* **%d** dice di stampare il primo *parametro* **x** come intero decimale
- Il secondo *specificatore* **%d** dice di stampare il secondo *parametro* **y**




```
$ ./printf_2
x  vale 5, y vale 2
```

Altri esempi con printf

- Stampa con minimo numero di cifre intere richieste

```
printf("x vale %d\n", x);
```

```
$ ./printf_4
x vale 7
```

- Stampa su, per es., 6 cifre intere con *padding* a sinistra

```
printf("x vale %6d\n", x);
```

```
$ ./printf_5
x vale      7
```

- Stampa su, per es., 6 cifre intere con *zeroing* a sinistra

```
printf("x vale %06d\n", x);
```

```
$ ./printf_5
x vale 000007
```

https://www.gnu.org/software/libc/manual/html_node/Formatted-Output.html

Assegnamento: riassunto e sintassi

- Inizializzazione alla dichiarazione *(Valore assegnato subito alla creazione della variabile)*

```c
int x = 10;
float y = 3.14;
```

- Assegnamento successivo *(prima si dichiara, poi si assegna)*

```c
int x;
x = 10;
```

- Assegnamento da input *(tramite funzioni come scanf)*

```c
int n;
scanf("%d", &n);
```

- Assegnamento multiplo *(possibile assegnare un valore a più variabili in un'unica istruzione)*

```c
int a, b, c;
a = b = c = 0;
```

Operatori di assegnamento composti

Il C consente di combinare un'operazione algebrica con un assegnamento, tramite gli operatori di assegnamento composti

Operatore	Espansione	Esempio in C
`+=`	`x = x + y`	`x = 5;` `x += 3; // x = 8`
`-=`	`x = x - y`	`x = 5;` `x -= 2; // x = 3`
`*=`	`x = x * y`	`x = 5;` `x *= 4; // x = 20`
`/=`	`x = x / y`	`x = 20;` `x /= 5; // x = 4`
`%=`	`x = x % y`	`x = 20;` `x %= 6; // x = 2`

Operatori di incremento e decremento



- Sintassi compatta per aumentare o diminuire una variabile intera di una unità.
- **Incremento**

```
x++; // equivalente a:  x = x + 1
```

- **Decremento**

```
x--; // equivalente a:  x = x - 1
```

- Li useremo come istruzioni di incremento/decremento
  (questi operatori sono anche espressioni di pre/post incremento e decremento, ma non tratteremo questo caso)